Here are a few things RSpec's suggested settings will do for you:

- Run RSpec without any changes to Ruby core classes (zero monkey-patch mode)
- Use the more verbose documentation formatter when you're running just one spec file
- Run your specs in random order

When you start a project with the `rspec --init` command (as we did in *First Steps*, on page 45), RSpec adds these suggestions to `spec/spec_helper.rb` but leaves them commented out. To enable them, find and delete the two lines marked `=begin` and `=end`.

Although all of these recommended settings are useful, we've actually turned two of them off to generate slightly shorter output for this book. The first one is the `warnings` setting for extra diagnostic info—great when you're writing a gem, but a little chatty when you're using a gem like Sequel that generates a lot of Ruby warnings.

05-unit-specs/01/expense_tracker/spec/spec_helper.rb

```
# config.warnings = true
```

We've also turned off `profile_examples`, a feature you already encountered in *Identifying Slow Examples*, on page 20:

05-unit-specs/01/expense_tracker/spec/spec_helper.rb

```
# config.profile_examples = 10
```

Getting a list of slow examples is helpful for large test suites, but we're not quite there with this project yet.

While we're in this file, add the following line inside the `RSpec.configure` block:

05-unit-specs/01/expense_tracker/spec/spec_helper.rb

```
RSpec.configure do |config|
  config.filter_gems_from_backtrace 'rack', 'rack-test', 'sequel', 'sinatra'
```

When a spec fails, the printed backtrace can contain dozens of lines of framework code, obscuring the application code you're looking for. RSpec already filters out its own code from the backtrace, and you can easily filter out other gems as well.[3] You'll have less to read, and you'll find errors more quickly.

3. http://rspec.info/documentation/3.6/rspec-core/RSpec/Core/Configuration.html#filter_gems_from_backtrace-instance_method